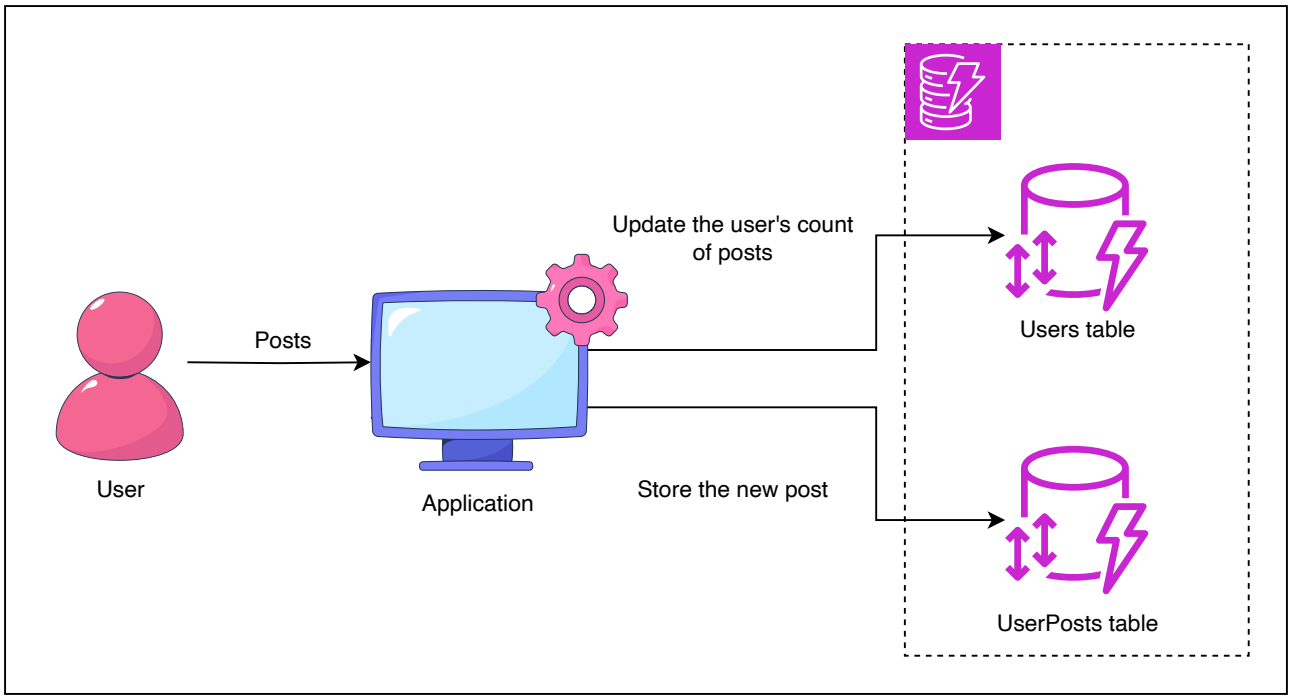
DynamoDB Transactions and Locking

Explore how DynamoDB supports atomic transactions and optimistic concurrency control to ensure data consistency. Understand the use of TransactWriteItems, TransactGetItems, conditional expressions, and idempotency for handling concurrent operations and preventing conflicts in distributed applications.

DynamoDB provides the `TransactWriteItems` and `TransactGetItems` API to enable atomic operations on multiple items across one or more tables. Each transaction operation succeeds or fails together, maintaining ACID properties (atomicity, consistency, isolation, durability).

For example, when a user publishes a new post, you need to create the new post in the `UserPosts` table and update the `Users` table to increment the `PostsCount` for that user. Both operations must succeed or fail together to maintain system integrity.

Ask



Updating UserPosts and Users table when a user posts

The `TransactWriteItems` and `BatchWriteItem`

`TransactWriteItems` is an atomic batch operation. It allows up to 25 actions in a single request, targeting one or more tables. If any action within the transaction fails because of a condition failure, throughput issue, or internal error, none of the actions are applied.

Another operation for writing multiple items to a DynamoDB table is `BatchWriteItem`, which allows up to 25 put or delete operations in a single call. It's efficient for bulk write scenarios where atomicity is not required, such as importing data or periodically syncing items.

> **Note:** A traditional `BatchWriteItem` performs best-effort writes without atomicity guarantees. On the other hand, `TransactWriteItems` ensures full atomicity: it's all or nothing.

Ask

Each `TransactWriteItems` request contains a list of `TransactItems`. Each `TransactItem` can be one of the following types:

Request Name	Description
Put	Insert a new item
Update	Modify an existing item
Delete	Remove an existing item
ConditionCheck	Verify a condition without modifying an item

Managing concurrency with conditional expressions

In highly concurrent systems, race conditions are inevitable. DynamoDB lets you mitigate this using *optimistic concurrency control* with *conditional writes*. We can specify conditions under which an update or delete should succeed.

Suppose two users try to purchase the last available item simultaneously. By setting a `ConditionExpression` that checks available stock, only one transaction will succeed. If the condition fails, DynamoDB throws a `ConditionalCheckFailedException`, signaling the client to retry or gracefully handle the conflict.

> **Key tip:** Using conditional writes inside a transaction can prevent lost updates and inconsistent states.

Ask

Concurrency control with conditional expression

Idempotency in DynamoDB

Idempotency ensures that making the same request multiple times has the same effect as making it once. For example, a user should post a social media post only once, even if they click the post button multiple times due to network errors.

DynamoDB handles idempotency by including `ClientRequestToken`—a unique, client-generated identifier for the transaction in the `TransactWriteItems` API. It uses this token to recognize duplicate transaction requests made within a 10-minute window and ensures only the first successful transaction is applied.

The `ClientRequestToken` can be used to add a new user post in the DynamoDB table:

main.py Python

```python
import boto3
import uuid
import json
from datetime import datetime, timezone

# Initialize the DynamoDB client
dynamodb = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

# Unique transaction ID
transaction_id = str(uuid.uuid4())
```

Ask

Python code for TransactWriteItems API with ClientRequestToken

If a duplicate request comes in with the same `ClientRequestToken` within that 10-minute window, the behavior is as follows:

- It simply returns the successful response again if the original transaction was successful.
- It waits and returns the final status if the original transaction is still in the processing state.
- It rejects the duplicate if the original request failed permanently.

Therefore, retries are safe and idempotent.

The `TransactGetItems` and `BatchGetItem`

`TransactGetItems` allows us to batch-read up to 25 items from one or more tables transactionally.
It guarantees that all the items are read consistently together, at the same time, without any other write operations modifying the data mid-read.

In simple terms, all the items we fetch either reflect the database state before any new writes that happen in parallel, or they fail.

In highly concurrent applications, normal `GetItem` or `BatchGetItem` operations can read data that is in the middle of being updated. If the application logic depends on multiple items being in a specific state together, using `TransactGetItems` is the safest way to avoid inconsistencies. We can use the API in the following cases:

- Reading both a user profile and their permissions concurrently
- Checking the stock of multiple products in the bucket before placing a combined order

Ask

- Fetching related records across multiple tables in a consistent snapshot

For example, a `TransactGetItems` API call can fetch the `post123` of `user456` from `UserPosts` and `userProfiles` table respectively:

main.py Python

```python
import boto3

dynamodb = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

Time_stamp = dynamodb.scan(
    TableName="UserPosts",
    FilterExpression='UserId = :uid', # Using UserId in KeyConditionE
    ExpressionAttributeValues={
        ':uid': {'S': "user456"}
    }
```

Python code for TransactGetItems API

Both of these items can be fetched in a separate get items command. However, suppose that while we are fetching the post author's data, the author changes the original content of the post. This leads to consistency issues.

Therefore, to avoid consistency issues, it is necessary to fetch both the author's profile and user posts from both tables at the same time.

Locking in DynamoDB

DynamoDB tries to minimize traditional locks. Instead of locking data like SQL databases, DynamoDB assumes that conflicts are rare and handles them o they happen. This approach is called optimistic concurrency control or optimistic locking.

Ask

Additionally, DynamoDB locks items when we use methods such as `TransactWriteItems` API, conditional writes, or strongly consistent reads. DynamoDB temporarily locks the item(s) involved to ensure that no two processes

Ask